

COS 214

Practical 5 - PDF

Done by : Senzo Lukhele (u24691497)

Rayhaan Ahmed (u25047762)

Rafael de Gouveia (u25054903)

CampusGuard System Description

When the fire alarm goes off in the Engineering Building, nobody has time for six subsystems arguing about whose job it is to unlock a door.

CampusGuard is our answer to that problem: a C++11 emergency response coordination platform built for a university campus, where security, medical, facilities, and access control all need to move together the moment an incident is reported, without any one of them needing to know how the others work.

The system follows one incident from the second it is logged to the moment it is resolved. An operator issues instructions as traceable **Commands** (dispatch a unit, lock down an area, sound the alarm, cancel an action) each one a self-contained object that can be executed, and where it makes sense, undone. Behind the scenes, an **IncidentCoordinator** acts as the campus's central nervous system: response teams never talk to each other directly, they report to the coordinator, and it decides who needs to know what. Security clears a scene, and medical is automatically put on standby. No team ever hardcodes a relationship with another.

Not every system on campus plays nicely, either. The ageing siren tower speaks its own primitive protocol, so an **Adapter** translates modern alert requests into whatever archaic signal it still understands - legacy infrastructure, quietly integrated rather than replaced. For the operator, the whole chaotic workflow of a real emergency response is wrapped behind a single **Facade** call, while every subsystem it touches remains fully usable on its own. An incident's **State** governs what can legally happen to it next, and alerts can be dynamically escalated in severity and scope through **Decorators**, stacked without a subclass explosion in sight.

GoF participant mapping for all six patterns

State Design Pattern

Context	Incident
State	IncidentState
ConcreteState	ResolvedState, EscalatedState, ReportedState, InProgressState, DispatchedState

Decorator Design Pattern

Component	DispatchOrder
ConcreteComponent	StandardDispatch
Decorator	DispatchDecorator
ConcreteDecorator	UrgentDispatchDecorator, BackupRequestDecorator

Command Design Pattern

Command	Command
ConcreteCommand	CancelActionCommand, DispatchUnitCommand, ActivateAlertCommand, LockdownAreaCommand
Invoker	OperatorConsole
Receiver	AlertChannel, AccessControlSystem, DispatchOrder, Incident
Client	

Adapter Design Pattern

Client	EmergencyResponseFacade
Target	AlertChannel
Adaptee	SirenTower
Adapter	SirenTowerAdapter

Fascade Design Pattern

Facade	EmergencyResponseFacade
Subsytem classes	OperatorConsole,

	AccessControlSystem, AlertChannel
--	--------------------------------------

Mediator Design Pattern

Mediator	IncidentMadiator
ConcreteMediator	IncidentCoordinator
Colleague	ResponseComponent
ConcreteColleague	SecurityTeam, MedicalTeam, FacilitiesTeam

Justification for the two team-selected patterns

State:

- An Incident moves through a defined lifecycle and what actions are legal at any point depends on which stage it is in. Such as, you are not supposed to respond to an incident that has been responded to, and you should not be able to dispatch one that has already been resolved. So that behaviour needs to be enforced, through using the current state an Incident is in.
- It helps keep Incident lightweight, as it does not need to know the full expectations for every state since it delegates to whatever the current state is.

Decorator:

- Alerts are not supposed to be one size fits all and the combination of qualities matters as much as any single one.
- A campus alert can be both urgent and campus wide at once, while a smaller incident might need neither or just one. While keeping the plain incident alert un-modified. This precisely fits the situation Decorator exists for.

GDB investigation and Valgrind evidence

```

CampusGuard shutdown complete.
==7892==
==7892== HEAP SUMMARY:
==7892==      in use at exit: 0 bytes in 0 blocks
==7892==    total heap usage: 73 allocs, 73 frees, 76,544 bytes allocated
==7892==
==7892== All heap blocks were freed -- no leaks are possible
==7892==
==7892== For lists of detected and suppressed errors, rerun with: -s
==7892== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

```

rayhaan@Rayhaans-MacBook-Air COS214-Practical-5 % docker compose run --rm campusguard gdb ./campusguard
Using host libthread_db library "/lib/aarch64-linux-gnu/libthread_db.so.1".
CampusGuard Emergency Response
=====
SCENARIO 1: Fire in Main Library (Facade)

FACADE: reportEmergency for INC-001 (Fire, severity 8) - status: Reported

Program received signal SIGSEGV, Segmentation fault.
0x0000aaaaaf1451f8 in BackupRequestDecorator::getBriefing[abi:cxx11]() const (
    this=0xaaaae8aeb7e0) at BackupRequestDecorator.cpp:13
13      return "BACKUP REQUESTED: " + wrapped->getBriefing();
(gdb) bt
#0 0x0000aaaaaf1451f8 in BackupRequestDecorator::getBriefing[abi:cxx11]() const (this=0xaaaae8aeb7e0) at BackupRequestDecorator.cpp:13
#1 0x0000aaaaaf1468bc in EmergencyResponseFacade::reportEmergency (
    this=0xfffffe69ad518, incident=0xfffffe69ad6e8, order=0xaaaae8aeb7e0,
    alertMessage="Fire evacuation : Main Library")
    at EmergencyResponseFacade.cpp:11
#2 0x0000aaaaaf14ab6c in main () at main.cpp:61
(gdb) frame 0
#0 0x0000aaaaaf1451f8 in BackupRequestDecorator::getBriefing[abi:cxx11]() const (this=0xaaaae8aeb7e0) at BackupRequestDecorator.cpp:13
13      return "BACKUP REQUESTED: " + wrapped->getBriefing();
(gdb) print wrapped
$1 = std::unique_ptr<DispatchOrder> = {get() = 0x0}
(gdb) print wrapped.get()
$2 = (DispatchOrder *) 0x0
(gdb) frame 2
#2 0x0000aaaaaf14ab6c in main () at main.cpp:61
61      facade.reportEmergency(&fire, fullOrder.release(), "Fire evacuation : Main Library");
(gdb) print standard
$3 = std::unique_ptr<DispatchOrder> = {get() = 0x0}
(gdb) print urgent
$4 = std::unique_ptr<DispatchOrder> = {get() = 0xaaaae8aeb7c0}
(gdb) print fullOrder
$5 = std::unique_ptr<DispatchOrder> = {get() = 0x0}
(gdb) quit
A debugging session is active.

    Inferior 1 [process 16] will be killed.

Quit anyway? (y or n) y
rayhaan@Rayhaans-MacBook-Air COS214-Practical-5 %

```

GitHub repository link

<https://github.com/RayhaanAhmed/COS214-Practical-5.git>

Team Contributions

Senzo:

- Compiled the PDF archive.
- Implemented the Command and Mediator Design Pattern sections of the system.
- Designed the sequence diagram for Command Design Pattern.

Rayhaan:

- Implemented the Adapter and Façade Design Pattern sections of the system.
- Design of the UML Diagram.
- Designed the sequence diagram for Adapter Design Pattern
- Docker, Main (building scenarios), and MakeFile compilation.

Rafael:

- Implemented the State and Decorator Design Pattern sections of the system.
- Designed the sequence diagram for the Adapter Design Pattern.
- Valgrind memory check.

Short collaboration reflection

This practical was a genuinely enjoyable one to work through as a team. We planned the design together before splitting up implementation along natural pattern boundaries, which kept everyone aligned on how the six patterns fit into one coherent system rather than working in isolation. Communication was consistent throughout, questions got resolved quickly, and everyone pulled their weight from the start till the end. It made a demanding practical feel genuinely manageable, and we are happy with what we built together.
